

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Project Report
on
”Ray Marching”

[Code No.: COMP 342]

Submitted by:

Bijan Bakabal Thapa (7)

Slok Pradhan (38)

Submitted to:

Mr. Dhiraj Shrestha

Department of Computer Science and Engineering

Submission Date:

August 4, 2026

1 Introduction

1.1 Background

Ray marching is a modern computer graphics rendering technique used to create three-dimensional (3D) scenes using mathematical equations instead of traditional polygon-based models. Unlike conventional rendering methods, ray marching traces rays from a virtual camera into a scene and repeatedly advances each ray by the distance returned from a Signed Distance Function (SDF). This process continues until the ray intersects the surface of an object or exits the scene. By representing objects mathematically, ray marching enables the generation of smooth surfaces and realistic visual effects such as lighting, shadows, and surface shading while requiring relatively simple geometric representations.

Recent advancements in programmable graphics hardware and shader programming have made ray marching an increasingly popular technique for procedural rendering and real-time graphics applications. Since scenes are generated mathematically rather than through complex polygon meshes, developers can create highly detailed procedural environments with comparatively compact code.

1.2 Motivation

Traditional rendering pipelines rely on polygon meshes to represent 3D objects, which often require significant modeling effort and additional memory for complex scenes. Ray marching offers an alternative approach by defining objects through Signed Distance Functions, allowing complex geometry to be described using mathematical expressions. This makes it particularly suitable for procedural graphics, visual demonstrations, and shader-based rendering.

The primary motivation behind this project is to gain practical experience in modern computer graphics concepts, GPU programming, and shader development. Implementing a ray marching renderer from scratch provides a deeper understanding of how rays interact with mathematical surfaces, how lighting is computed, and how real-time rendering is achieved using OpenGL and GLSL.

1.3 Objectives

The objectives of this project are as follows:

- Develop a real-time ray marching application using the C programming language.
- Implement Signed Distance Functions (SDFs) for rendering basic 3D objects such as spheres, boxes, and planes.
- Generate realistic lighting effects including surface normal estimation and shadows using GLSL shaders.
- Allow users to interact with the rendered scene through camera movement and viewing controls.

- Study the performance and efficiency of ray marching as an alternative rendering technique for procedural graphics.

1.4 Scope

This project focuses on implementing the core concepts of ray marching using C, OpenGL, GLFW, GLEW, and GLSL. The application renders basic procedural objects defined through Signed Distance Functions and demonstrates fundamental graphics techniques including object intersection, surface normal estimation, lighting, shadow computation, and camera navigation.

The scope of the project is limited to real-time rendering of simple procedural objects and does not include advanced features such as global illumination, physically based rendering, volumetric rendering, or animation systems. Nevertheless, the implementation provides a solid foundation for understanding modern shader-based rendering techniques and can be extended in future work to support more advanced computer graphics applications.

2 Project Description

2.1 Technology Stack

The project is developed using the C programming language together with OpenGL and its supporting libraries for real-time graphics rendering. C was chosen because of its high performance, low-level memory management, and efficient interaction with graphics APIs, making it well suited for graphics applications.

OpenGL serves as the primary graphics API responsible for rendering the scene. GLFW is used to create the application window, manage the OpenGL context, and handle keyboard and mouse input. GLEW (OpenGL Extension Wrangler Library) provides access to modern OpenGL functions required by the application. The rendering algorithm is implemented using GLSL (OpenGL Shading Language), where the ray marching process executes inside the fragment shader.

2.2 System Architecture

The application consists of two major components: the CPU-side application written in C and the GPU-side shader program written in GLSL. The CPU is responsible for initializing the OpenGL environment, creating the application window, processing user input, and sending rendering parameters to the GPU.

The GPU performs the computationally intensive ray marching calculations. Camera parameters and other required values are passed to the fragment shader through uniform variables. The fragment shader then generates the final colour for every pixel, producing the rendered image displayed on the screen.

2.3 Rendering Pipeline

Unlike traditional rendering techniques that depend on polygon meshes, the renderer generates objects procedurally using mathematical functions. For every pixel on the screen, a ray is projected from the virtual camera into the scene. The ray advances according to the distance returned by the Signed Distance Function until it either intersects an object or reaches the maximum marching distance.

The rendering pipeline follows these steps:

1. Initialize the OpenGL environment.
2. Create the application window using GLFW.
3. Load OpenGL functions using GLEW.
4. Compile and link the GLSL shaders.
5. Generate viewing rays from the virtual camera.
6. Execute the ray marching algorithm.
7. Compute surface normals.
8. Apply lighting and shadow calculations.
9. Display the rendered image.

2.4 Signed Distance Functions

Signed Distance Functions (SDFs) are used to represent every object in the scene mathematically. Rather than storing vertices and polygons, each object is defined by a function that returns the minimum distance between any point in space and the object's surface.

A positive distance indicates that the point lies outside the object, a negative value indicates that it lies inside the object, and a value of zero represents the object's surface. This mathematical representation allows the ray marching algorithm to determine safe step sizes while traversing the scene.

The application implements Signed Distance Functions for basic geometric objects including spheres, boxes, and planes.

2.5 Ray Marching Algorithm

Ray marching is the core rendering technique used in this project. Starting from the camera position, each ray repeatedly advances through the scene by the distance returned from the nearest Signed Distance Function. Since the distance represents the minimum safe movement before reaching an object, large empty regions can be traversed efficiently without unnecessary calculations.

The algorithm terminates when one of the following conditions is satisfied:

- The ray reaches the surface of an object.
- The maximum marching distance is exceeded.
- The maximum number of marching iterations is reached.

After an intersection is detected, the renderer computes the appearance of the surface using normal estimation and lighting calculations.

2.6 Lighting and Shading

Lighting calculations are performed after determining the point of intersection between the ray and an object. Surface normals are estimated numerically by sampling the Signed Distance Function at neighbouring points. These normals determine how light interacts with the surface.

The renderer implements diffuse lighting and shadow calculations to improve depth perception and provide a more realistic appearance. These calculations are performed entirely within the GLSL fragment shader, allowing every pixel to be processed efficiently by the GPU.

2.7 Camera Controls

The application allows users to explore the rendered scene through camera movement. GLFW processes keyboard and mouse input, enabling users to move the camera and observe objects from different viewpoints. Interactive camera controls help demonstrate the three-dimensional nature of the scene and allow the rendered objects to be examined from multiple perspectives.

2.8 Application Workflow

The overall workflow of the application is summarized below:

1. Initialize GLFW and create the application window.
2. Initialize GLEW and configure OpenGL.
3. Load and compile the vertex and fragment shaders.
4. Define Signed Distance Functions for the scene objects.
5. Generate rays from the camera through each screen pixel.
6. Execute the ray marching algorithm in the fragment shader.
7. Calculate surface normals, lighting, and shadows.
8. Display the rendered frame.
9. Repeat the rendering process until the application exits.

2.9 Project Snapshots

The following figures present the implementation and output of the ray marching application.

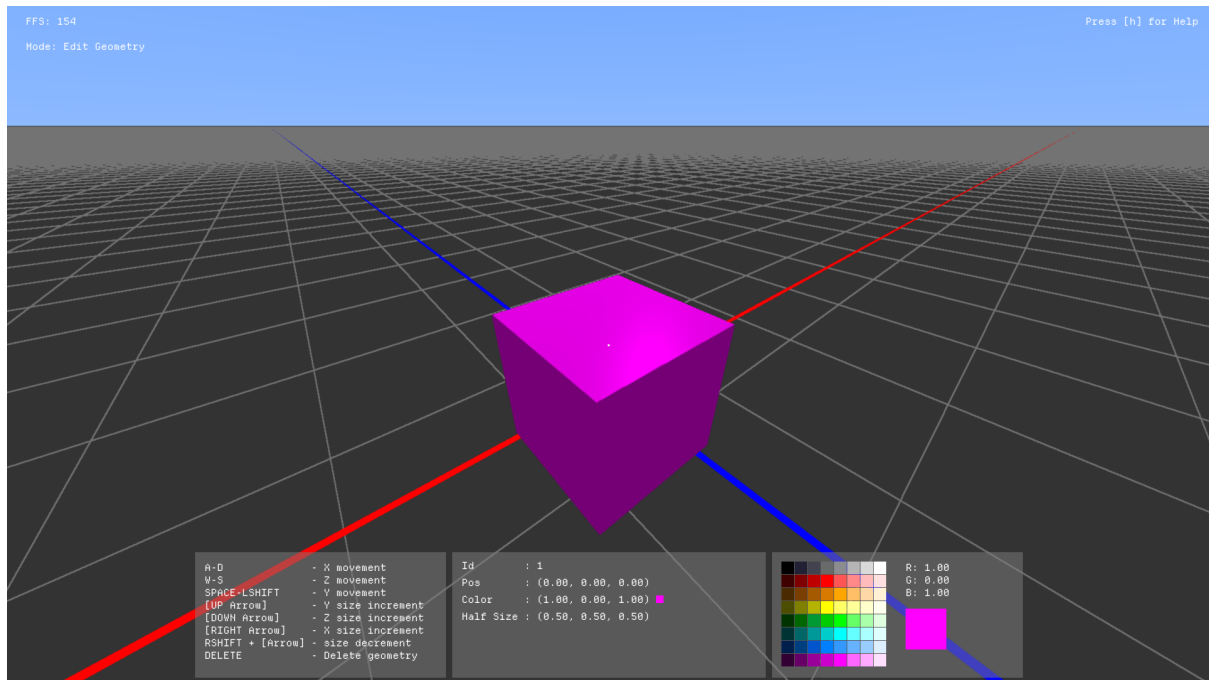


Figure 1: Controls for the geometry.

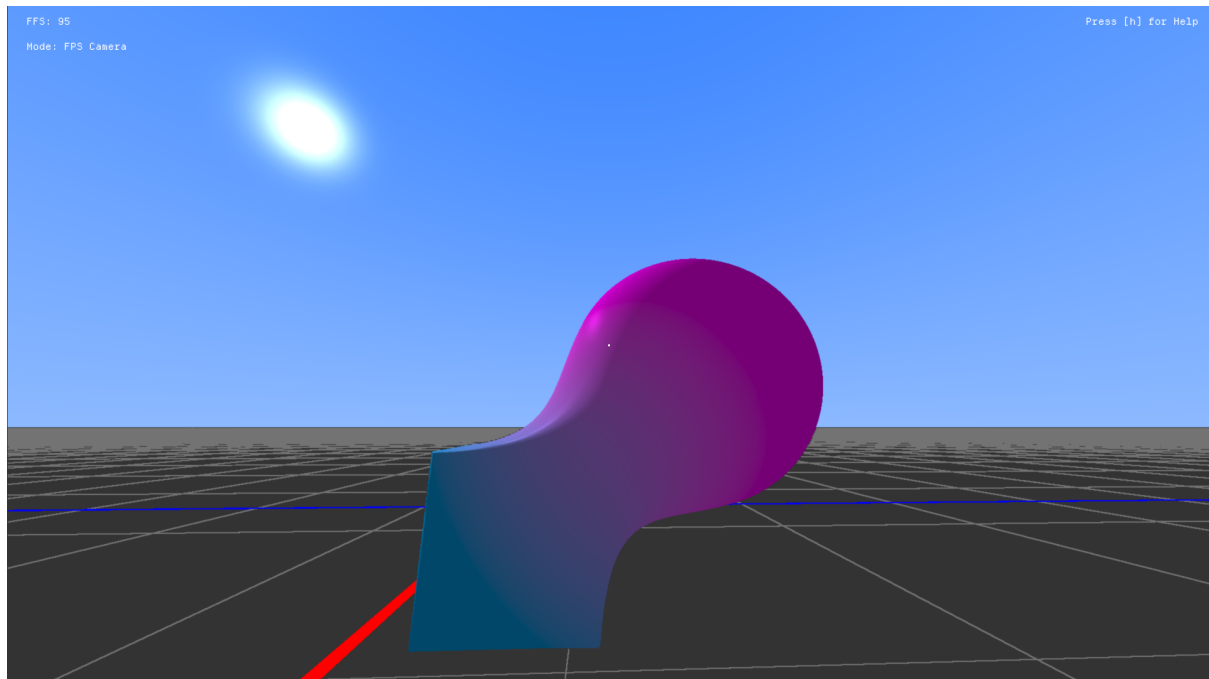


Figure 2: Rendering using union operation.

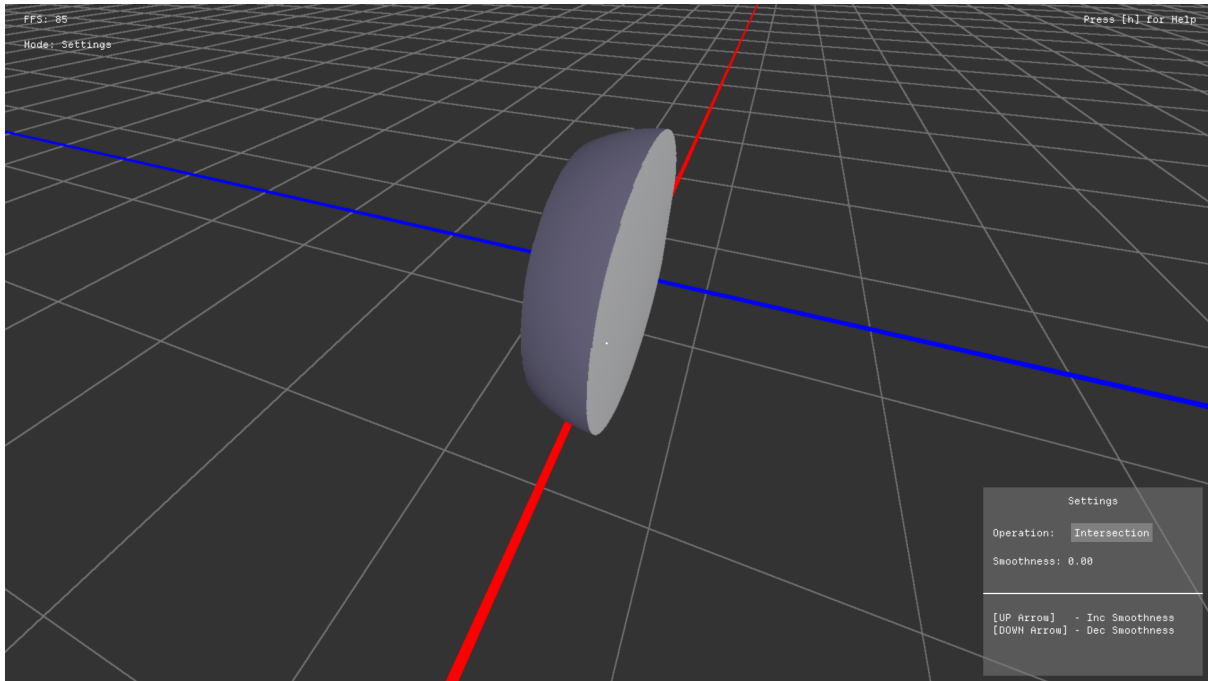


Figure 3: Rendering using intersection operation.

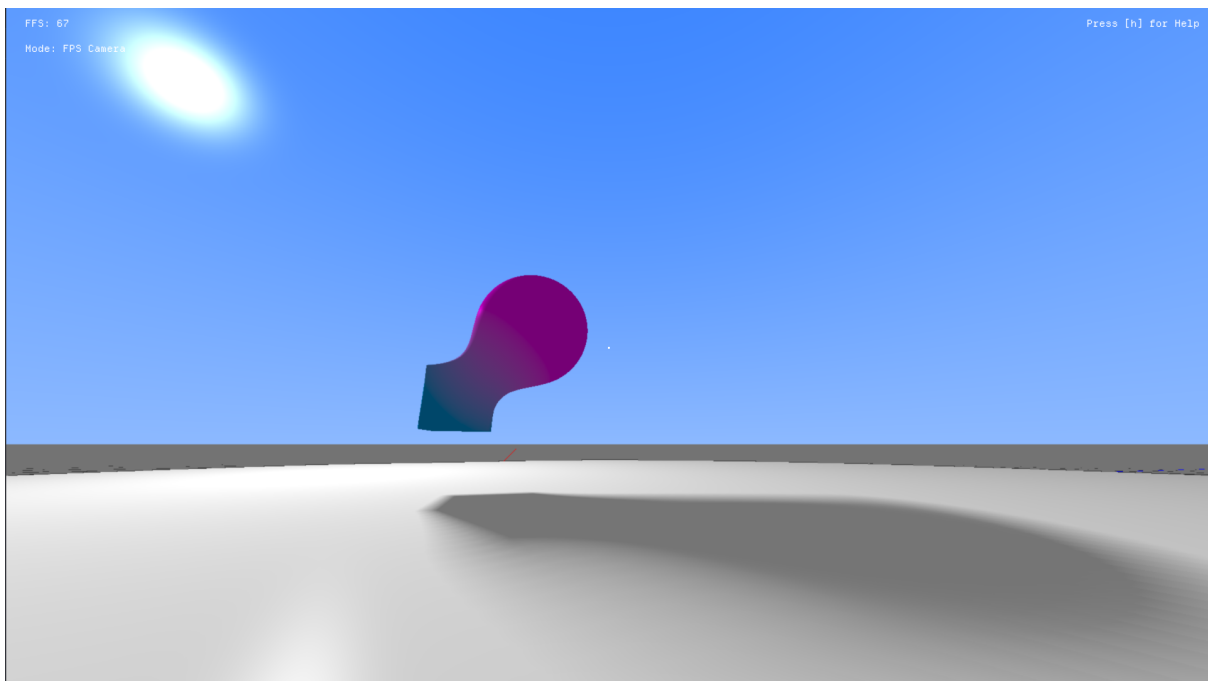


Figure 4: Lighting and shadow effects produced using ray marching.

3 Conclusion

This project successfully demonstrates the implementation of a real-time ray marching renderer using the C programming language together with OpenGL, GLFW, GLEW, and GLSL. Instead of relying on conventional polygon-based rendering techniques, the application generates three-dimensional objects using Signed Distance Functions (SDFs) and renders them through the ray marching algorithm executed within the fragment shader. This implementation demonstrates how mathematical representations can be used to efficiently render procedural scenes in real time.

Throughout the development of the project, practical knowledge was gained in modern computer graphics concepts, including OpenGL programming, shader development, Signed Distance Functions, surface normal estimation, lighting, shadow computation, and camera control. Implementing these components from scratch provided a deeper understanding of GPU-based rendering and the procedural techniques used in real-time graphics applications.

The completed application is capable of rendering basic geometric primitives, such as spheres, boxes, and planes, while supporting interactive camera movement for exploring the generated scene. Although the project focuses on the fundamental principles of ray marching, it establishes a solid foundation for understanding shader-based rendering and procedural modeling.

Several improvements can be considered for future development. These include implementing Constructive Solid Geometry (CSG) operations for creating more complex procedural models, supporting additional Signed Distance Functions for new geometric primitives, improving lighting models, incorporating anti-aliasing techniques, and optimizing the rendering algorithm to achieve better performance in more complex scenes.

Overall, the project successfully achieved its objectives by implementing a functional real-time ray marching renderer and demonstrating the practical application of mathematical modeling, GPU programming, and modern rendering techniques in computer graphics.

4 References

1. de Vries, J. (n.d.). *LearnOpenGL*. Retrieved August 3, 2026, from <https://learnopengl.com/>
2. GLFW Developers. (n.d.). *GLFW Documentation*. Retrieved August 3, 2026, from <https://www.glfw.org/documentation.html>
3. GLEW Developers. (n.d.). *The OpenGL Extension Wrangler Library*. Retrieved August 3, 2026, from <http://glew.sourceforge.net/>
4. Quilez, I. (n.d.). *Distance Functions*. Retrieved August 3, 2026, from <https://iquilezles.org/articles/distfunctions/>
5. The Khronos Group. (n.d.). *OpenGL Shading Language (GLSL)*. Retrieved August 3, 2026, from https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language
6. The Khronos Group. (n.d.). *OpenGL Documentation*. Retrieved August 3, 2026, from <https://www.opengl.org/documentation/>
7. Wong, J. (2016, July 15). *Ray Marching and Signed Distance Functions*. Retrieved August 3, 2026, from <https://jamie-wong.com/2016/07/15/ray-marching-signed-distance-functions/>